

조준희 포트폴리오

카카오페이 데이터 엔지니어 · 데이터 플랫폼 지원

데이터 엔지니어링 · 플랫폼 운영

프로필

데이터를 수집하고 정리하는 과정부터 서비스에서 활용하기까지의 흐름을 설계하고 개선합니다. DocSuri의 데이터 파이프라인과 인프라를 운영하며 해결한 문제, 개인 프로젝트에서 처리 결과를 검증한 경험을 소개합니다.

corpseonthemission@icloud.com github.com/revenantonthemission

대표 프로젝트

프로젝트 01

DocSuri

4인 팀에서 여러 출처의 논문을 수집·구조화·임베딩·색인하는 파이프라인과 인프라를 담당했습니다. 대량 백필 중 발생한 검색 503 오류를 복구하고, 사용자 열람용 문서를 생성하는 큐와 워커를 분리했습니다.

문제

DocSuri는 AI/ML 논문을 탐색하고 원문에 근거한 한국어 요약을 제공하는 서비스입니다. 수집한 논문은 검색, 문서 열람, 요약에 사용됩니다. 출처가 다른 논문의 중복과 버전, 철회 여부를 처리하는 한편, 과거 데이터를 대량으로 채우는 백필이 사용자 응답을 방해하지 않도록 해야 했습니다.

백필이 검색과 문서 열람에 미친 영향

2026년 7월 AWS에서 운영하던 중, 대량의 arXiv 백필이 OpenSearch 조회에 영향을 주면서 k-NN 검색이 2초 제한을 넘겨 타임아웃으로 실패하고 503 오류를 반환했습니다. 사용자 열람용 문서를 생성하는 작업도 약 3.3만 건의 백필 뒤에서 대기했습니다. 각 프로세스가 요청 제한을 지켜도 워커를 3개 실행하면 전체 요청량은 늘어난다는 점을 확인했습니다.

역할

AWS AI School 2기 4인 팀의 데이터 엔지니어로서 논문 코퍼스(U1)의 수집·구조화·임베딩·색인, 인프라 운영과 후속 개선을 맡았습니다. 검색·요약·에이전트의 초기 구현과 이후 추가한 GROBID TEI 구조 파서는 다른 팀원이 담당했습니다.

데이터를 주고받는 규칙과 협업

수집 데이터의 필드나 의미를 바꿀 때는 공용 JSON Schema와 Python DTO뿐 아니라, 그 데이터를 사용하는 요약 기능과 프론트엔드도 함께 수정했습니다. 호출 동작은 Protocol 인터페이스로 분리하고, 데이터를 만드는 쪽과 사용하는 쪽의 작업 및 검토 책임을 정리했습니다. 기존 문서 추적성 도구에는 한국어 요구사항 ID, 모노레포 경로, 유닛 계획을 해석하는 기능을 보완해 검토해야 할 문서 간 연결을 찾을 수 있도록 했습니다.

핵심 결정

수집량을 제한하고 사용자 작업을 분리

7월 1일에는 수집 워커를 3개에서 1개로 줄여 검색을 복구하고, 사용자 DocModel 작업에 전용 우선순위 큐를 마련했습니다. 7월 2일에는 별도 워커 서비스로 옮겨 실행 자원도 분리했습니다. 외부 데이터 소스의 요청 한도에 맞춰 확장 상한을 정하고, bulk 작업은 30분, DocModel 작업은 5분 이상 대기할 때 알람이 울리도록 했습니다. 실패 메시지를 보관하는 DLQ의 재처리 절차에는 검색 지연도 함께 확인하도록 명시했습니다.

여러 출처의 논문을 하나의 레코드로 통합

DOI, arXiv ID, 정규화한 제목·첫 저자·연도 순으로 식별 키를 선택하고, 출처별 우선순위에 따라 대표 레코드를 정했습니다. 우선순위가 낮은 중복 데이터는 원문 추출과 임베딩 전에 걸러냈으며, 대표 레코드에는 같은 논문의 다른 식별자와 발견한 출처를 남겼습니다. 버전과 우선순위 조건에 따라 레코드를 갱신하고, 논문이 철회되면 청크·캐시·자산과 대표 레코드의 상태를 정리하도록 구성했습니다.

재시도와 완료 상태를 구분

같은 버전의 메시지가 다시 전달되거나 bulk 색인의 일부 항목이 실패하는 경우를 테스트했습니다. 항목이 실패하면 완료로 기록하지 않고, 재시도 메시지와 DLQ에는 출처와 실패 단계를 남겼습니다. 출처별 처리 지점인 watermark가 뒤로 이동하지 않고 서로 독립적으로 관리되는지는 속성 테스트로 확인했습니다. 인덱스를 재구축할 때는 스케줄·이벤트 등록과 기존 작업의 실행을 제한했습니다.

변경 목적에 맞춘 재처리

저장 구조가 바뀌면 기존 레코드를 복사하고, 모델이 바뀌면 저장된 청크를 다시 임베딩하며, 파서가 바뀌면 캐시에 보관한 원본 바이트를 다시 파싱하도록 처리 시작점을 나눴습니다. 열람용 DocModel을 다시 만드는 작업과 검색용 청크·벡터를 갱신하는 작업도 구분해 불필요한 원문 재수집을 줄이고자 했습니다.

모델 호출량을 조절하고 중단 지점부터 재시작

계정 전체에 적용되는 모델 호출 한도를 고려해, 예상 토큰 수에 따라 배치의 실행 간격을 조절했습니다. 속도 제한 모드에서는 대상 인덱스에 이미 기록된 문서를 조회해 재임베딩을 생략했습니다. 토큰 버킷의 용량보다 큰 배치도 처리할 수 있는지는 회귀 테스트로 확인했습니다. 기존 결과를 재사용하려면 같은 대상 인덱스와 설정으로 재시작해야 합니다.

새 인덱스로 전환하고 되돌리는 절차

새 인덱스에 데이터를 적재한 뒤 최소 문서 수와 원본 대비 수량을 점검하고, replica와 refresh 설정 복원, alias 전환, 수집 재개를 각각 수행하도록 절차를 나눴습니다. alias에서 이전 대상을 제거하고 새 대상을 추가하는 작업은 한 요청으로 처리합니다. 벡터 차원이 바뀔 때는 읽기·쓰기 서비스가 공유하는 스펙과 실행 이미지를 함께 맞추고, 복구할 때는 이전 이미지와 alias로 되돌리도록 문서화했습니다.

외부 문서를 가져올 때 입력 검증

접속 대상의 공인 IP와 리다이렉트를 검사하고, 스트리밍 응답 크기, XML 엔티티 확장, 압축 해제에 대한 제한을 보완했습니다. 6월 29일 기록의 크기 상한은 외부 응답 64MB, XML 32MB, 압축 파일의 개별 항목 20MB와 함께 200MB입니다. 이 결과는 외부 문서를 받아들이는 단계의 구현과 회귀 검증에 해당합니다.

서비스 예산과 사용자별 사용량을 별도로 관리

팀이 만든 공통 비용 제어 기능을 바탕으로, 에이전트가 warning과 critical 상태를 처리하는 방식을 보완하고 실행 비용을 관측 지표에 기록했습니다. Redis 기반 사용자 쿼터를 연결해 서비스 전체 예산과 사용자별 사용량 제한을 구분했습니다.

구조

논문 수집부터 검색·열람·요약까지

arXiv, Semantic Scholar, OpenAlex에서 가져온 논문을 수집 큐에 넣고, 문서 구조화와 청크·임베딩 생성을 거쳐 OpenSearch에 색인했습니다. 이 데이터를 검색·열람·요약에 연결했습니다. AWS 운영 당시에는 Python, PostgreSQL, SQS, ECS/Fargate, S3, Bedrock, CDK, CloudWatch, Redis를 사용했으며, 사용자 열람용 문서 생성은 별도 큐와 워커로 처리했습니다.

본문과 검색 청크의 기준이 되는 DocModel

문서의 섹션과 블록을 공통 구조로 표현하고, 이를 바탕으로 읽기 순서에 맞춘 본문과 검색 청크를 만들었습니다. 문단·표·수식·그림 캡션에는 논문·버전·섹션·블록 식별자를 연결하고, 검색 토큰과 원문 블록을 참조하는 blockRefs를 분리했습니다. 저장된 데이터의 호환성은 파서와 스키마 버전으로 판단하며, 이를 사용하는 기능의 파생 캐시에서도 DocModel 세대를 구분합니다.

문서 벡터와 질의 벡터의 호환 조건

팀의 공용 스펙은 모델, specVersion, 차원, 거리 함수, 정규화 방식을 같은 벡터 공간을 구성하는 조건으로 정의하고, 문서용과 질의용 input type을 구분합니다. 스펙 비교 함수와 각 필드의 불일치 테스트를 사용했지만, 이 검사가 모든 실행 환경에서 배포를 차단하도록 강제되는 것은 아닙니다.

검색 색인과 그림·도표 생성을 분리

팀의 그림·도표 파이프라인은 주 색인 처리가 끝난 뒤 별도로 실행하며, 성공을 필수 조건으로 삼지는 않습니다. 객체 저장소에 먼저 저장한 뒤 DB의 자산 목록인 manifest를 갱신하고, 문서 블록의 assetId와 연결합니다. 검색할 수 있는 상태와 화면에 표시할 자산이 준비된 상태를 구분하며, 버전이 바뀌거나 논문이 철회되면 파생 자산을 정리합니다.

사용자가 업로드한 PDF의 처리 규칙

공개 논문과 구별할 수 있도록 userdoc 식별자와 별도 작업 종류를 정의했습니다. 작업 정보, 소유자, 참조 레코드가 서로 일치하는지 검사하고 저장된 PDF를 공통 DocModel로 변환했습니다. 잘못된 입력이나 파싱할 수 없는 문서는 재시도로 해결되지 않는 영구 실패로 분류해 DLQ로 보냈습니다. 이는 작업 메시지를 검증하는 규칙이며, 읽기 API에서 소유자를 확인하는 문제까지 해결한 것은 아닙니다.

AWS에서 로컬 서버로 운영 환경 이전

2026년 8월 AWS 운영을 종료하고, 로컬 서버의 컨테이너와 MinIO·ElasticMQ·Ollama를 사용하도록 연결을 변경했습니다. 외부 heartbeat로 호스트 장애를 관측하고, launchd의 실행 권한 때문에 백업 정리가 실패하는 문제는 manifest, 날짜 기준 보존 정책, 재시도를 통해 보완했습니다. 현재는 단일 호스트에서 운영하는 구성입니다.

배포 설정과 실제 동작을 대조

자산 조회에 필요한 S3 prefix 범위의 권한을 보완하고, 지표의 namespace에 맞는 IAM 권한과 실제 지표 전달 여부를 대조했습니다. 기존 큐를 다시 생성하려다 충돌하는 것을 확인한 뒤 import 방식으로 가져왔으며, 기능 플래그는 보존했습니다. 커밋 태그, 서비스 안정화 상태, health 검사로 배포를 추적했습니다. 사용이 끝난 도메인의 기본값을 정리할 때도 재현용 테스트 데이터와 식별용 namespace는 남겼습니다.

수집한 데이터를 사용하는 팀의 기능

제가 맡은 파이프라인의 데이터가 검색·요약·개인화·인용 기능에서 어떻게 쓰이는지도 함께 살폈습니다. 아래 내용은 팀이 구현하고 평가한 사례로, 해당 기능 전체를 제가 구현했다는 의미는 아닙니다.

중복 이벤트와 실패 원인을 구분

검색 이력에서는 requestId를 사용해 같은 이벤트가 다시 전달된 경우와 사용자가 같은 질의를 새로 실행한 경우를 구분합니다. 임베딩 장애에 대응해 어휘 기반 검색으로 대체한 상태와 색인을 조회할 수 없는 상태도 나눴습니다. 요약 기능 역시 비용 제한, 생성 장애, 원문 부재를 각각 별도의 상태로 다룹니다.

캐시 재사용 조건과 개인화 가중치 제한

파생 캐시는 논문, 버전, 언어, persona, 용어집, 모델, 프롬프트, DocModel 세대를 구분합니다. 개인화에 사용하는 가중치는 개별 값뿐 아니라 절댓값의 합에도 상한을 뒀습니다. 세 범주의 점수가 같아지는 반례는 속성 테스트의 고정 사례로 추가했습니다.

요약 근거 평가와 인용 그래프의 한계 표시

요약의 근거를 평가할 때는 실제 논문 8건에서 발췌한 수치로 18개 사례를 만들고, 잘못 통과시킨 경우(false-pass)와 불필요하게 판단을 보류한 경우(false-abstain)를 구분했습니다. confident로 분류된 14개 사례의 성공 기록은 이 통제된 평가 범위에 한정됩니다. 인용 그래프는 대표 식별자(canonical ID)를 찾지 못한 경우, 순환과 중복, 탐색량 제한, 부분 결과를 표현하도록 설계했습니다.

결과

장애를 재현한 질의로 검색 복구 확인

7월 1일 워커를 3개에서 1개로 줄인 뒤, 장애를 재현했던 질의가 정상 응답하는 것을 확인했습니다. 다음 날 추가로 안정화하면서 큐를 보존한 채 수집과 자동 확장을 일시 정지했습니다. 이때 검색은 기능 저하(degraded) 상태가 아닌 정상 결과 20건을 반환했습니다. 아래 수치는 당시 문서와 커밋에 남아 있는 관측값입니다.

논문 한 편을 대상으로 재처리 검증

단일 논문 파일럿에서 118개 청크와 13개 자산을 생성했습니다. 처리 대상 큐의 잔량은 0이었고, 수집 DLQ는 111건으로 유지됐으며 검색에도 degraded 표시가 없었습니다. 7월 8일 문서 생성 DLQ가 24건에서 0건이 된 것은 이와 다른 큐의 후속 복구 기록입니다. 이 결과가 전체 백필의 완료나 장기적인 SLA 충족을 뜻하지는 않습니다.

혼합 부하에서 측정한 응답 시간과 오류율

조치 후 가상 사용자 20명(20 VU)으로 실행한 k6 기록에서 검색 p95는 664.9ms, 전체 HTTP 오류율은 0.01%였습니다. 프론트엔드, readyz, 검색 요청을 섞은 테스트이며, 하나의 질의를 반복했으므로 캐시의 영향이 있을 수 있습니다. 오류율 0.01%도 검색 요청만의 수치는 아닙니다. 이전에 측정한 검색 p95 9,061ms는 요청 12회·동시성 3이라는 다른 조건에서 나온 값이므로, 두 결과로 개선율을 계산하지 않았습니다.

실제 운영 설정과 코드의 차이 확인

별도의 작업 적체 조사에서는 약 15.3만 건이 대기하는 상황에서 코드와 템플릿의 확장 상한은 10이지만 실제 설정은 0이라는 차이를 확인했습니다. 이는 워커가 실행되지 않는 원인을 좁혀 간 기록입니다. 관측 모듈에는 중복 이벤트를 추적하는 상태의 메모리 상한과 로그의 민감 정보 가림 처리를 보완했습니다. 다만 오래된 ID를 제거한 뒤에는 같은 이벤트를 다시 받을 가능성이 남습니다.

환경별로 확인한 백업과 복구 범위

AWS 운영 당시에는 팀이 CloudWatch 이메일 수신을 확인하고, RDS snapshot을 새 인스턴스로 복원해 available 상태가 되는 것을 확인했습니다. 로컬 환경으로 옮긴 뒤에는 launchd로 백업을 실행하고, 로컬과 iCloud 보관 경로에 dump 파일이 생성된 것을 확인했습니다. dump가 생성됐다는 확인은 복원 시험과 다릅니다. AWS에서 수행한 검증 또한 현재 객체 저장소와 검색 인덱스를 함께 복원할 수 있다는 보장은 아닙니다.

배운 점

작업의 성공과 데이터의 활용을 각각 검증

작업이 성공했는지, 결과가 저장됐는지, 데이터 품질이 적절한지, 사용자가 실제로 활용할 수 있는지는 따로 확인해야 합니다. 재처리와 온라인 조회의 목표를 나누고, 처리량뿐 아니라 실패가 미치는 범위, 복구 방법, 데이터를 사용하는 쪽의 요구까지 함께 살피는 기준을 얻었습니다.

테스트로 확인한 범위를 넘어서 보장하지 않기

중복 전달과 부분 실패를 테스트했다고 해서 모든 동시 실행에서 exactly-once 처리나 저장소 간 원자성이 보장되는 것은 아닙니다. 인덱스 점검은 수량 부족을 감지하는 수준이며, 전환 절차 자체가 검증 실행을 강제하지도 않습니다. 재파싱은 개별 작업이 실패해도 종료 코드가 0일 수 있으므로 로그와 결과를 대조해야 합니다. 자산 저장 역시 분산 트랜잭션으로 묶여 있지는 않습니다.

아직 해결해야 할 데이터 품질과 접근 권한 문제

최신 감사에서는 코퍼스에 테스트용 데이터가 섞인 문제, 공유 캐시가 해당 원문과 정확히 연결되는지의 문제, 사용자 문서의 읽기 권한 문제가 남아 있었습니다. 일부 타입 생성 작업의 실패를 상위 단계에 전달하는 처리와 빈 결과를 degraded 상태로 표시하는 문제도 확인했습니다. 후속 검증에서는 원본과 색인의 ID·내용 일치, 실패 건수와 종료 상태, 출처 식별과 소유권에 따른 접근 범위, 호스트 장애 이후의 복원을 확인 기준으로 삼았습니다.

앞으로 더 깊이 다루고 싶은 문제

여러 출처의 데이터를 정규화하고 결합한 경험, 다시 시작할 수 있는 배치를 만든 경험, 읽기·쓰기 서비스의 데이터 규칙과 부하·비용·복구를 다룬 경험을 더 큰 데이터 플랫폼으로 확장하고 싶습니다. 새로운 기술을 선택할 때도 데이터를 만드는 과정과 이를 사용하는 동료의 요구를 함께 살피겠습니다.

관련 자료

[DocSuri 팀 GitHub 저장소 보기](#) [DocSuri 개인 fork 저장소 보기](#) [검색 복구를 위한 수집량 제어](#)

[파이프라인·워커·인프라 개선 PR #323](#) [외부 요청량 제한 개선 PR #420](#)

obsidian-custom-publish

Obsidian 문서를 Rust·Astro 기반 정적 블로그로 게시하는 프로젝트입니다. CI가 성공했는데도 다이어그램이 누락되는 문제를 수정해, 렌더링이 실패하면 배포 과정에서도 실패를 감지하도록 했습니다.

문제

Obsidian 문서의 위키링크, 검색, 탐색 트리, 미리보기, 다이어그램을 정적 사이트에서 사용할 수 있도록 만드는 개인 프로젝트입니다. 2026년 7월에는 CI가 성공으로 끝났지만, 28개 글에서 Mermaid 렌더링 65건이 실패해 다이어그램이 빠진 문서가 배포됐습니다.

렌더링 실패가 CI에 전달되지 않은 원인

로컬 브라우저 설정 파일이 Git 관리 대상에서 제외돼 CI 환경으로 전달되지 않았고, Puppeteer는 브라우저를 찾지 못했습니다. 렌더러 오류가 경고로만 처리되면서 전처리 명령은 성공으로 끝났고, 배포도 그대로 진행됐습니다.

역할

저장소 소유자로서 Rust 전처리기, Astro의 렌더링 흐름, 배포 구성을 설계하고 구현했으며 운영도 맡았습니다.

핵심 결정

문서 스캔, 링크 처리, 변환, 검색 데이터 생성, 출력으로 이어지는 전처리를 Astro 렌더링과 분리했습니다. CI에는 브라우저 실행 경로를 명시하고, Mermaid·D2의 SVG 렌더링이 실패하면 이를 집계해 전처리 명령도 실패로 종료하도록 바꿨습니다. 원인을 조사하는 데 필요한 생성 파일은 남겼습니다.

구조

Rust CLI가 Obsidian Vault를 가공해 콘텐츠와 검색·그래프·탐색 데이터를 만듭니다. Astro 6 사이트는 이 결과를 받아 unified·rehype·Shiki·KaTeX 파이프라인으로 정적 HTML을 생성합니다.

결과

브라우저 설정 유무에 따른 재현 결과

브라우저 경로를 설정하지 않으면 렌더링 65건이 실패하고 종료 코드 1을 반환했습니다. 경로를 설정하면 종료 코드 0을 반환하고 밝은 테마와 어두운 테마용 SVG 130개가 생성됐습니다. 이는 해당 렌더링 과정을 검증한 결과이며, 모든 파일 쓰기 실패를 확인했다는 의미는 아닙니다.

입력 규칙과 실제 생성 파일을 함께 검증

출력 전에 공개 범위와 홈페이지 참조 규칙을 검사하고, manifest와 실제 생성 파일 목록을 대조하는 테스트를 뒀습니다. 외부 다이어그램 렌더러를 제외한 테스트 데이터에서는 같은 입력으로 다시 실행했을 때 출력이 바이트 단위로 일치하는지도 확인합니다.

배운 점

명령이 성공했는지와 실제 산출물이 올바른지는 함께 확인해야 한다는 점을 배웠습니다. 렌더러의 실패를 파이프라인 종료 상태에 반영하고, 입력 규칙과 출력 결과를 검증해 같은 과정을 재현할 수 있도록 배포를 관리합니다.

관련 자료

[obsidian-custom-publish GitHub 저장소 보기](#) [렌더링 실패를 CI로 전파한 수정](#)

mcp-local-reference

로컬 Zotero 라이브러리에서 자료를 찾고 PDF 본문·도판을 추출하며, Harvard 형식의 인용을 제공하는 MCP 서버를 만들었습니다.

문제

연구 자료를 외부 서비스에 복제하지 않으면서, 로컬 Zotero 데이터와 PDF에서 필요한 내용을 빠르게 찾고 인용할 수 있는 형태로 꺼내고 싶었습니다.

역할

저장소 소유자로서 MCP 도구를 설계하고, 로컬 데이터 접근부터 인덱싱과 인용 출력까지의 흐름을 구현했습니다.

핵심 결정

Zotero의 SQLite 데이터베이스와 로컬 파일을 읽어 메타데이터 검색과 의미 기반 검색 인덱스를 결합했습니다. 원본 데이터와 검색 인덱스는 분리하고, 자료 조회·도판 추출·인용 생성은 각각 별도의 도구로 제공했습니다.

구조

Python MCP 서버가 Zotero 메타데이터와 컬렉션, PDF 본문·도판, 의미 기반 검색 인덱스를 연결하고 클라이언트의 요청에 결과를 반환합니다.

결과

하나의 로컬 서버에서 서지 메타데이터 검색, 컬렉션 탐색, PDF 본문·도판 추출, Harvard 형식의 인용 생성, 라이브러리 인덱싱을 제공합니다.

배운 점

저장된 자료를 사용자가 찾고 참조할 수 있는 형태로 가공해 본 경험입니다. 원본의 구조와 출처를 유지하면서도 활용 목적에 맞게 검색하고 조회할 수 있도록 인터페이스를 구성하는 데 집중했습니다.

관련 자료

[mcp-local-reference GitHub 저장소 보기](#)

AdiuBear

음성·이미지·텍스트 입력을 처리하는 Flutter 앱을 만들고, Gemini와 연결하는 Cloud Run 미들웨어를 구현했습니다.

문제

모바일 앱에서 음성·이미지·텍스트를 Gemini에 전달하되, API 키가 클라이언트에 노출되지 않도록 해야 했습니다.

역할

저장소 소유자로서 Flutter 클라이언트와 Gemini 연동을 구현하고, 두 시스템을 연결하는 방식을 반복해서 검토했습니다.

핵심 결정

API 키를 보호하기 위해 Cloud Run 미들웨어를 두었습니다. Live API도 실험했지만 응답을 파싱하는 방법에 관한 문서가 충분하지 않아 서버를 통한 호출 방식으로 되돌렸습니다.

구조

Flutter 앱에서 음성·이미지·텍스트 입력을 받고, Firebase·Vertex AI SDK 또는 Cloud Run 중계 계층을 통해 Gemini를 호출하는 구조입니다.

결과

여러 형태의 입력을 처리하는 흐름과 API 키를 보호하는 중계 계층을 구현했습니다. Live API를 실험한 결과를 바탕으로 서버 호출 방식으로 전환했습니다.

배운 점

API가 제공하는 기능만으로 연동 방식을 결정하기는 어렵다는 점을 배웠습니다. 키 보안, 응답을 파싱하는 데 필요한 문서, 실제로 운영할 수 있는지도 함께 검토해야 했습니다.

관련 자료

[AdiuBear GitHub 저장소 보기](#)